

magicmaus tutorial — a dummy walkthrough

This tutorial runs the whole magicmaus pipeline on a tiny **8-methyl toy dataset** you can read end to end. By the finish you will know how to build the three input files, run the engine, read every field of its output, and see the intensity lever break a degeneracy in front of you.

magicmaus assigns methyl NMR peaks to the methyls of a known structure by fusing two ideas — **MAUS** (SAT hard constraints that bound the answer and never exclude the truth) and **MAGIC** (intensity-weighted NOE scoring that commits to a single best answer). See [README.md](#) for the theory; this is the hands-on version.

0. Setup

```
python3 -m venv .venv
.venv/bin/pip install python-sat numpy
```

Everything below is run from the repository root with `.venv/bin/python`.

1. The dummy “protein”

Our toy structure has **8 methyls** on 7 residues. Two of them, the Leu-40 $\delta 1/\delta 2$ pair, are deliberately given **identical chemical shifts** — a symmetry no achiral NOE network can break. Generate the dataset:

```
.venv/bin/python make_dummy.py
# wrote examples/dummy/ (8 methyls, 56 directed NOE cross peaks)
```

This writes five files into `examples/dummy/`:

file	role
<code>model.pdb</code>	the structure (methyl-carbon coordinates)
<code>hmqc.tsv</code>	the 2D peak list — the input (anonymous peak ids)
<code>hmqc_true.tsv</code>	the answer key (which methyl each peak really is)
<code>noesy.tsv</code>	3D (H)CCH NOE cross peaks, boolean (no intensity)
<code>noesy_intensity.tsv</code>	same peaks, with realistic $1/r^6$ intensities

The HMQC input

`hmqc.tsv` is `label → H_ppm → C_ppm → res_type`. The `label` (`P1`, `P2`, ...) is an anonymous id that leaks nothing about the answer; `res_type` is just the amino acid one-letter code:

label	H_ppm	C_ppm	res_type	
P1	1.300	19.000	A	
P2	1.420	20.500	A	
P3	0.800	13.000	I	
P4	0.750	24.000	L	<- Leu-40 61 and 62 share
P5	0.750	24.000	L	<- the SAME shifts (a real symmetry)
P6	0.900	21.000	V	
P7	0.950	22.500	V	
P8	1.100	21.800	T	

Note P4 and P5 are indistinguishable by shift — remember them.

The NOESY input

noesy.tsv is label → C1 → C2 → H2 , one row per 3D (H)CCH cross peak. The observed methyl is matched by (H2, C2) ; its NOE partner contributes carbon C1 **only** (a 3D experiment does not resolve the partner's proton). That carbon-only partner is the root of most ambiguity.

2. First run — boolean NOESY

Run magicmaus, pass the answer key with `--truth` so it scores itself, and write a per-peak table with `--out` :

```
.venv/bin/python magicmaus.py \
  examples/dummy/model.pdb examples/dummy/hmqc.tsv examples/dummy/noesy.tsv \
  --truth examples/dummy/hmqc_true.tsv --out dummy_calls.tsv
```

```
methyls(G nodes)=8  HMQC peaks=8  NOESY cross peaks=56
NOE match (tol H+-0.02/C+-0.2): firm=30 ambiguous(dropped)=26 unmatched=0
--- MAUS envelope (never excludes truth) ---
unique(1 option)      = 2/8
ambiguous(2-3 options)= 6/8
--- magicmaus commitment (single coherent call) ---
confidence: unique=2  scored=4  ambiguous=2
--- scored vs truth ---
truth in MAUS option set = 8/8 = 100.0% (never-exclude guarantee preserved)
magicmaus single call   = 2/8 = 25.0% correct
  unique    : 2/2 = 100.0% correct
  scored    : 0/4 =  0.0% correct
  ambiguous : 0/2 =  0.0% correct
```

Two things to read here:

- **truth in MAUS option set = 8/8 = 100 %** — the envelope. For every peak, the true methyl is somewhere in its reported option set. This is guaranteed, always, no matter how ambiguous the data.
- **single call = 25 %** — the committed single answer is mostly wrong. On a *boolean* network the scoring has nothing to grade, so the four **scored** peaks are coin-flips. That is the honest baseline. Section 4 fixes it.

Reading the per-peak table

```
column -t -s $'\t' dummy_calls.tsv
```

label	call	confidence	n_options	options	truth	call_correct	truth_in_set
P1	A20CB	scored	2	A10CB,A20CB	A10CB	0	1
P2	A10CB	scored	2	A10CB,A20CB	A20CB	0	1
P3	I30CD1	unique	1	I30CD1	I30CD1	1	1
P4	L40CD2	ambiguous	2	L40CD1,L40CD2	L40CD1	0	1
P5	L40CD1	ambiguous	2	L40CD1,L40CD2	L40CD2	0	1
P6	V50CG2	scored	2	V50CG1,V50CG2	V50CG1	0	1
P7	V50CG1	scored	2	V50CG1,V50CG2	V50CG2	0	1
P8	T60CG2	unique	1	T60CG2	T60CG2	1	1

The **confidence** column is the key. magicmaus never just guesses silently — it tells you *how much to trust each call*:

tier	meaning	in the dummy
unique	one option survives the hard constraints — forced, certain	P3 , P8
scored	several options; the NOE score prefers this one	P1 P2 P6 P7
ambiguous	a tied alternative exists — a genuine symmetry, a coin flip	P4 , P5

P4 / P5 are the Leu-40 $\delta 1/\delta 2$ pair we made shift-degenerate. They are **truly unresolvable** without stereospecific labeling, so magicmaus flags them **ambiguous** (margin 0) and reports *both* members in **options** — it refuses to pretend. **truth_in_set** = 1 for every row: the envelope holds even where the single call is wrong.

3. Why the scored peaks flip on boolean data

P1 / P2 (the two Alas) and P6 / P7 (the two Vals) each have two options and are coupled by NOEs. On a boolean network both members of a pair have the same *pattern* of contacts, so the score is a near-tie and the pick is arbitrary — here it lands on the wrong one. The information that would break the tie is *how strong* each NOE is: a strong NOE means a close contact. That is exactly what an intensity column carries.

4. Second run — with intensities

noesy_intensity.tsv has a 5th **intensity** column ($1/r^6$, as a real experiment would). Point magicmaus at it — nothing else changes:

```
.venv/bin/python magicmaus.py \
  examples/dummy/model.pdb examples/dummy/hmqc.tsv examples/dummy/noesy_intensity.tsv \
  --truth examples/dummy/hmqc_true.tsv
```

```
confidence: unique=2 scored=4 ambiguous=2
magicmaus single call    = 6/8 = 75.0% correct
  unique   : 2/2 = 100.0% correct
  scored   : 4/4 = 100.0% correct    <- all four now correct
  ambiguous: 0/2 =  0.0% correct
```

The four `scored` peaks flip from **0/4** → **4/4**. Feeding intensities lifts the run from **25 %** → **75 %** — and the envelope is still 100 %. The two remaining misses are only the genuinely-symmetric Leu pair, which no data short of stereo labeling can resolve.

NOESY	single-answer correct	what moved
boolean	25 %	only the 2 forced <code>unique</code> peaks
+ intensities	75 %	all 4 <code>scored</code> peaks now resolve

This is the whole point of the synthesis in miniature: **MAUS keeps the truth in every option set; MAGIC's intensity scoring then commits correctly within that bounded space.** Neither half does this alone.

5. Options you will actually use

flag	default	when to change it
<code>--tol-h / --tol-c</code>	0.02 / 0.20	tighten to match sharper, well-referenced shifts (fewer ambiguous matches → more <code>unique</code>)
<code>--soft-ambiguous</code>	off	real NOESY with intensities — reuses the ambiguous cross peaks MAUS discards as extra tie-breakers
<code>--hmbc FILE</code>	—	HMBC geminal-link peak list, to couple each Leu/Val methyl pair
<code>--out FILE</code>	—	write the per-peak table (always do this)
<code>--short-cut / --long-cut</code>	6 / 10 Å	structural contact-distance classes

Tentative anchors: if you already know a few assignments, write them into the `res_type` cell of `hmqc.tsv` (e.g. `I30D1` instead of `I`) — magicmaus pins that peak and the constraint propagates through the NOE network to sharpen the rest.

6. On your own data

1. Build `hmqc.tsv` (peak picks + residue types) and `noesy.tsv` (3D (H)CCH cross peaks; add the 5th intensity column if you have it) in the formats above.
2. Provide a `model.pdb`.
3. Run:

```
.venv/bin/python magicmaus.py model.pdb hmqc.tsv noesy.tsv --out calls.tsv
```

(drop `--truth` — you do not have it on real data).

4. Read `calls.tsv`: trust `unique` calls, treat `scored` as strong leads, and carry `ambiguous` peaks as their full option set until other data resolve them.

To build peak lists straight from a BMRB shift file plus a PDB, see the sibling [../maus/make_peaklists.py](#). For a realistic 192-methyl example (maltose-binding protein) and the full three-way comparison against MAGIC and MAUS, see [README.md](#).